

The Next-Generation Lattice zkVM: Research and Implementation Roadmap

What to implement next for prover and verifier efficiency, 50–100 KB proofs, privacy, distributed proving, and soundness — a prioritized pipeline toward 100 million proven RISC-V cycles per second, with the SALSAA and Twist-and-Shout completion plan, the GPU acceleration program, and the Bend / Lean formal-verification tracks.

LZX Engineering

github.com/idrees2516/lzx — 31 crates, ~66,000 lines of pure-std Rust, post-quantum under Module-SIS

Table of Contents

Chapter 1 - Executive Summary	1
Chapter 2 - Current State of the LZX System	2
Chapter 3 - The SOTA zkVM Landscape, 2025-2026	5
Chapter 4 - LambdaClass, ZisK, and the GPU-Native Prover Roadmap	7
Chapter 5 - The Next Papers: A Prioritized Implementation Pipeline	9
Chapter 6 - Hitting 100 Million Cycles per Second	12
Chapter 7 - Hitting the 50-100 KB Proof Envelope	14
Chapter 8 - GPU Acceleration in Rust	15
Chapter 9 - Formal Verification: Bend, HVM, and What They Can and Cannot Do	17
Chapter 10 - Soundness, Completeness, and Security Program	18
Chapter 11 - Immediate Implementation: Completing SALSAA and Twist-and-Shout	20
Chapter 12 - Phased Roadmap and Success Criteria	22

Chapter 1 - Executive Summary

This report answers one question in depth: **what should the LZX lattice zkVM implement next** so that prover time, verifier time, proof size, privacy, distributed proving, and above all soundness, completeness, and security all move decisively toward the stated engineering targets - 100 million RISC-V cycles per second of proving throughput and a 50-100 KB proof envelope. The analysis is grounded in a line-by-line audit of the current repository state (31 crates, roughly 66,000 lines of pure-std Rust, 1,136 passing tests, seventeen research papers implemented end-to-end) combined with a survey of the 2025-2026 state of the art across SP1, Jolt, RISC Zero, Valida, OpenVM, Pico, Nexus, ZisK, and the LambdaClass ecosystem, together with the newest folding, IVC, PCD, and distributed-proving literature.

The central finding is that **LZX has already implemented essentially all of the cryptography it needs** for a post-quantum, compact-proof zkVM at the paper level: the LaBRADOR and Greyhound engines with the 53 KB accounting at 2^{30} , the ring-lookup layer (ePrint 2026/471), the LatticeFold+ norm controls (2026/721), ZK-PCD from accumulation schemes (2026/289), holography PCD (2026/538), LatticeBlindFold (2026/1857), CauchyFold high-arity folding (2026/2011), the streaming small-space prover (2025/611), and the monomial-basis sumcheck (2026/762). What separates the system from its performance and security targets is therefore **not missing papers but unfinished integration and engineering**: the dense-prover cap that limits programs to 2^{12} cycles, the missing instruction-semantics constraint families, the missing sparse prover in which zero entries are free, the missing second-level fold that closes the binding gap, and the absence of GPU kernels.

The recommended program, in priority order, is: **(1)** complete SALSAA and Twist-and-Shout to their paper-faithful production form - the sparse prover and the constraint families are the single highest-leverage items in the entire backlog, because they simultaneously unlock scale, prover speed, and the deletion of re-execution from the verification path; **(2)** land the Wave 8 engineering set - Goldilocks AVX-512/IFMA kernels, the 8-prime RNS crate for LaBRADOR polynomial arithmetic, and the sumcheck constant-factor pass; **(3)** build the GPU prover in Rust (cudarc backend with tensor-core matrix multiplication for the Ajtai MAC layer and radix-butterfly NTT kernels); **(4)** implement distributed proving by sharding the cycle trace and folding shard proofs via the in-tree CauchyFold/Symphony machinery plus the distributed-SNARK-by-folding blueprint of ePrint 2026/1995; **(5)** wire privacy in through LatticeBlindFold and the ZK-PCD accumulation layer rather than bolting on ad-hoc blinding; and **(6)** run the security program - production parameter instantiation gated by the SIS estimator, QROM composition attestations, and the two-level fold - before any external claim of 128-bit security.

On formal verification, the report is deliberately honest: **Bend (HigherOrderCO) is a parallel functional language on the HVM optimal runtime, not a proof assistant**. The correct use of Bend in this project is as a high-assurance, automatically parallel executable specification layer - a second independent implementation of the RV64IMAC semantics and the algebraic identities for differential testing at scale. Machine-checked theorems about soundness-critical lemmas (norm budgets, challenge-space operator norms, fold identities) should be written in Lean 4, with Bend-HVM used to

execute specifications and the existing 1,136-test Rust corpus as the ground truth. Both tracks are specified in chapter 9 with concrete phases.

33.0 KB current batched-compact memory-argument proof (fibonacci, 185 cycles)	49x labinius reference-round end-to-end speedup after the AVX-512 parity wave	53 KB Greyhound evaluation-proof accounting at $N = 2^{30}$, implemented in-tree
---	---	---

Every claim in this report is traceable either to a file in the repository, to a section of an ePrint paper, or to a cited public source. Chapter 2 establishes the audited baseline; chapters 3 and 4 survey the competitive landscape and the LambdaClass/ZisK roadmap; chapter 5 is the prioritized paper pipeline; chapters 6 and 7 are the throughput and proof-size engineering budgets; chapters 8 and 9 cover GPU acceleration and formal verification; chapter 10 is the security program; chapter 11 is the immediately actionable completion plan for SALSAA and Twist-and-Shout; and chapter 12 binds everything into a phased roadmap with success criteria.

Chapter 2 - Current State of the LZX System

2.1 What is implemented

LZX is a from-scratch, production-oriented implementation of the modern lattice-based zero-knowledge stack in pure-std Rust with zero external dependencies. The workspace is organized as 31 crates covering the full hierarchy: the Goldilocks field and transcript substrate (lattice-core), negacyclic NTT and ring arithmetic (lattice-ring), Ajtai Module-SIS commitments with a cached-NTT fast path and zero-skipping MACs (lattice-commitment), the virtual-polynomial sumcheck engine with Spartan-style zerocheck (lattice-sumcheck, lattice-projsumcheck), CCS relations (lattice-relations), the folding family (lattice-folding: ProtogaLattice, LatticeFold+, Cyclo, PikkuFold, Symphony, SuperNeo, Neo, PGL), the lookup family (lattice-lookup, lattice-lookup-ring), the SALSAA response layer (lattice-salsa), the PCS backends (lattice-pcs, lattice-akita, lattice-rokoko, lattice-labinius, and the LaBRADOR/Greyhound engine in lattice-greyhound and lattice-labrador), memory and VM proving (lattice-memory, lattice-vm, lattice-zkvm), streaming (lattice-streaming), PCD (lattice-pcd, lattice-holo), blinding (lattice-blindfold, lattice-zk), norm-checking (lattice-ttrp, lattice-labinius), folding-specific crates (lattice-cauchyfold, lattice-accordion), and the SIS security estimator (lattice-sis-estimator).

Seventeen papers are realized end-to-end with prover, verifier, and exact algebraic identity tests, including the two that define the compactness target: the LaBRADOR engine (ePrint 2022/1341) and Greyhound (ePrint 2024/1293) are implemented paper-faithfully at the papers' $q = 2^{32}-99$, including the Figure 2/3 protocols, the CWSS extractor of Lemma 3.2, the recursion machinery of section 5.3 and the tail of section 5.6, the R1CS reductions of section 6, and the Table 4 parameter accounting that yields **53**

KB evaluation proofs at $N = 2^{30}$. The ring-lookup layer (2026/471) runs over the CRT-split lattice ring itself and is wired into the zkVM's v3 pipeline as the memory argument. The LatticeFold+ ell2-norm checks of 2026/721 (JL projection relation of knowledge, exact-shortening RoK, no-drift norm ledger) are implemented in lattice-folding/lfplus_l2.rs.

2.2 Measured performance

The reproducible benchmark matrix measures the end-to-end memory-argument proof: nine Twist-and-Shout instances (four register limbs, four RAM limbs, one fetch Shout), 13 sumcheck legs each, roughly 120 legs total, with virtual-Val evaluation, matrix-evaluation sumchecks for every one-hot claim, and two Ajtai bundles authenticating every base claim. Verification never re-executes the program; the verifier recomputes only public tables and $O(\lambda)$ field work. The compact pipeline (docs/DESIGN_50KB.md) then replaced the Theta(N) digit-revealing bundle openings with the folded amortized opening, and the Stage-4 leg batching restructured approximately 117 per-instance legs into 12 staged, dependency-ordered batched sumchecks.

Program	Cycles	Clear mode	Compact mode	Batched compact	Compact prove	Compact verify
fibonacci	185	3,640 KB	75.5 KB	33.0 KB	1,429 ms	196 ms
regex	451	7,218 KB	108.4 KB	~105 KB (est.)	4,600 ms	288 ms
muldiv	392	7,212 KB	~107 KB	~104 KB (est.)	4,831 ms	295 ms
memory_ops	2,831	not proved	not proved	not proved	dense-prover cap	dense-prover cap
sorting	12,648	not proved	not proved	not proved	dense-prover cap	dense-prover cap

Table 2.1 - End-to-end zkVM memory-argument benchmarks (release build)

The batched-compact composition for fibonacci is: batched sumcheck legs approximately 6 KB (12 batched sumchecks, down from 55 KB across 108 legs), evaluation claims 3.5 KB, column commitments approximately 16 KB after the $k = 4$ estimator-driven hardening, carriers 0.8 KB, compact openings approximately 6 KB, and the statement 0.5 KB. At the kernel level the labinius AVX-512 backend runs the upstream kernel designs natively (vertical batch-of-32 binary NTT kernels with vpermb lookup tables and lazy reduction, the vpmaddwd raw-accumulation commitment MAC, PCLMULQDQ binary-field arithmetic), verified bit-exact against the scalar reference: the reference round runs 49x end-to-end (3,885 ms to 79 ms) with the proof 1.63x smaller (915 KB to 560 KB), sumcheck proving is 2.9x faster, LaBRADOR ring operations are 2.2-2.4x faster, and kernel-level speedups reach 270x (forward NTT), 436x (commitment MAC), and 66x (carry-less multiply).

2.3 The three binding constraints

First, the dense-prover cap. The twist legs materialize $K \times T_s$ matrices prover-side, so the harness only proves programs with T at most 2^{12} cycles and a RAM window of at most 256 words. This is precisely the asymptotic wall that the Twist-and-Shout paper exists to break: the paper's sparse provers (sections 6.3 and 7 - the doctrine that zero entries are free) remove the cap and are the Wave 8.5 follow-up. Until the sparse prover lands, every throughput number in this report is a kernel-scale projection, not a system-scale measurement, and this honesty matters when comparing against zkVMs that prove 2^{20} -cycle programs today.

Second, the binding-level soundness gap in the compact fold. The estimator-run MSIS table (SECURITY.md, Stage 5.1) published an honest verdict: the single-level fold at $k = 2$ has approximately 2^{12} binding security at every response length. The interim hardening ($k = 4$, amplitude 2^6) doubles the column commitments and tightens the relaxed bound 64x and is shipped; the sound posture still needs the second-level fold (Stage 5.2). This is the difference between a research artifact and a system that can make a 128-bit claim, and it is scheduled before any performance work that would have to be redone under different parameters.

Third, the missing instruction-semantics constraints. Wave 7.4 landed the memory argument without re-execution (columns, ledger, memory, memproof - 552 workspace tests), but the arith/logic/comparison/control/routing/halted constraint families are still staged substrate rather than implemented constraints (constraints.rs). Until those families exist, the prove path covers memory but not the full decode-execute semantics of the RV64IMAC ISA, which is what a zkVM is for. Chapter 11 treats this as part of the immediate implementation plan.

2.4 The Wave 8 backlog as it stands

Item	Wave	Content	Why it matters
gen_* transforms + vertical AuxData	8.1	labinius fold/verify kernel families (2,642 upstream LOC)	reference round 222 to ~100 ms; removes the 170 MB scatter
Goldilocks AVX-512 kernels	8.2	IFMA/VPCLMULQDQ kernels for field-bound stages	4-8x on sumcheck/commit/eval hot paths
lattice-rns 8-prime crate	8.3	RNS arithmetic for LaBRADOR polx + RoKoko CRT	10-50x on the large-modulus proving path
bin_large + fusion + prefetch	8.4	labinius encode kernels (910 upstream LOC)	encode stage parity with upstream
Sumcheck constant-factor pass	8.5	single-binding rounds, in-place fix_variables, Gruen/Dao-Thaler	2-4x on every sumcheck in the system
ZK wiring into the zkVM	8.6	per-stage blinding (Blindfold pattern), sparse masking, simulator KATs	privacy: the zkVM is currently not zero-knowledge
QROM attestations + grinding	8.7	regenerate from the real stage list; machine-checked grinding bounds	Fiat-Shamir security accounting

Item	Wave	Content	Why it matters
Production parameters	8.8	estimator-gated, digest-pinned SecurityProfiles	honest 128-bit posture
Chunked-chain recursion	8.9	shared-phi chunked chains with carry gadgets	labinius constraint count down; recursion depth control
Four production modules	8.10	Akita planner/validator; HyperWolf compaction; Symphony CP-SNARK; Quasar IVC	the L-effort paper completions

Table 2.2 - Wave 8 designated components (docs/papers/unimplemented)

Chapter 3 - The SOTA zkVM Landscape, 2025-2026

3.1 The field

The zkVM field has consolidated around a small number of architectural patterns, and the differences between them explain most of the performance spread. **STARK-based systems** (RISC Zero with FRI and DEEP-ALI composition; SP1 built on Plonky3 with its FRI-based `Plonky3` backend) hash commitments and thus inherit transparency and post-quantum conjectures, at the price of larger proofs (tens to hundreds of KB) and hash-heavy provers. **Multilinear-plus-sumcheck systems** (Jolt's lookup-centric PIOP over its Spartan-lineage backend; Pico from Brecht Devos, built on a multilinear PCS with Spartan-style folding and an exceptionally fast verifier; OpenVM's modular design with a system of composable extensions and precompiles) target the sumcheck-native regime where prover work is dominated by field multiplications over the trace. **Folding-first systems** (Pico v2 and the Neo/Protogalaxy lineage; Valida, which pairs a formally specified ISA with a Plonky3-based prover) amortize the witness through folding and then decide once. **GPU-native systems** (ZisK from the LambdaClass orbit; Nexus's proving network) treat the prover as a massively parallel GPU workload first and a protocol second. LZX is unique in this landscape: it is the only system whose entire stack - commitments, PCS, folding, lookups, PCD - is lattice-based and post-quantum under Module-SIS rather than discrete-log or hash-only assumptions.

System	Proof stack	Commitment/PCS	Prover posture	Proof size	Distinguishing trait
SP1 (Succinct)	Plonky3 / FRI, recursion + wrapping Groth16	hash-based (FRI), Baby Bear field	CPU SIMD, precompiles, shardable	~100-300 KB wrapped	most complete production tooling; Plonky3 kernels
Jolt (a16z)	lookup-centric PIOP, Spartan-lineage sumcheck	multilinear (Gemini-style) PCS	CPU, ~6x speedup wave; zkSecurity audits found significant bugs	~200-500 KB	RISC-V decode made into lookups; fastest CPU proving class

System	Proof stack	Commitment/P CS	Prover posture	Proof size	Distinguishing trait
RISC Zero	STARK (FRI, DEEP-ALI), Groth16 wrap	hash-based	CPU/GPU, most consistent cross-machine results	~100-200 KB	mature; Veridise-audited; concurrency model
Valida	Plonky3-based, formally specified ISA	hash-based	CPU SIMD	large (STAR K-class)	formal ISA spec; the gold standard for verifier-facing cleanliness
OpenVM	modular extensible zkVM	multilinear + lookup	CPU, precompile ecosystem	~100-300 KB	composable custom instructions (transcendental ops, ECC)
Pico / Pico v2	Spartan + folding, multilinear PCS	multilinear	very fast prover, tiny verifier cost	~100-400 KB	folding amortization; verifier in the ~ms class
Nexus	zkVM + proving network	multilinear	distributed proving network + NEX economics	variable	network-native; prover market
ZisK (LambdaClass orbit)	GPU-native prover, Cairo lineage design taste	hash/field mix	GPU-first, real-time proving goal	tens of KB	Baylina school: prover as a GPU program; deployed via Aligned
Ethrex + Zoda	Ethereum block proving, multi-zkVM integration	per-zkVM	SP1 / RISC Zero / ZisK / OpenVM backends	per backend	verification-layer integration patterns worth copying
LZX (this repo)	lattice sumcheck + folding + PCD stack	Ajtai MSIS commitments, La BRADOR/Greyhound/labinius PCS	pure-std CPU SIMD today; GPU planned	33 KB (kernel scale) / 53 KB accounting at 2^{30}	post-quantum under Module-SIS; zero-dependency reproducibility

Table 3.1 - SOTA zkVM comparison (public information, 2025-2026)

3.2 What the landscape teaches

Precompiles are the dominant prover economics. Every production zkVM funnels hashing, elliptic curves, and transcendentals through hand-written constraint modules rather than general ISA semantics; SP1's SHA-256 and Keccak precompiles and OpenVM's extension system are the pattern. For LZX the analog is a precompile surface keyed to the ring-native operations it is unusually good at: NTT-flavored transforms, big-integer multiplication via CRT splitting, and Keccak-f1600 (the substrate already contains a bit-exact implementation). A precompile is worth implementing only when its Rust guest usage frequency justifies the constraint surface; the benchmark harness already measures this per program class.

Memory-argument design decides the prover constant. Jolt's read-write matrix machinery and Twist-and-Shout's one-hot columns are the two ends of a spectrum; ZisK's school treats the memory proof as the GPU kernel first. LZX's ring-lookup composition (2026/471) is competitive on paper with

both, but its dense materialization forfeits the advantage - which is exactly why the sparse prover (chapter 11) is the highest-leverage item in this report rather than a new paper.

Security incidents are the field's real teacher. ZK Security's 2025-2026 quarterly series reported significant soundness bugs found in Jolt; Veridise's audit of RISC Zero (July-December 2024) reported 20 issues and 8 suggestions; the 2026 SoK on zkVM understanding catalogs the recurring failure classes: under-constrained circuits, Fiat-Shamir domain-separation errors, lookup edge cases (empty tables, duplicate reads), and parameter drift between paper and code. LZX's countermeasures are already unusually strong - hard norm gates that refuse to fold rather than wrap silently, a QROM query ledger with attestations, digest-pinned KATs, a 4,000-mutation envelope fuzz corpus, differential ISA testing, and chi-square zero-knowledge simulators - and chapter 10 extends them with production parameters and the second-level fold. The lesson to import from the field is process: every new protocol module lands with its negative tests (tamper suites) in the same commit as its positive tests, which the repo already enforces for the Greyhound engine and should enforce everywhere.

Benchmarks in the field are non-comparable by default. The Fenbushi standardized benchmarking study (August 2025) found RISC Zero the most consistent and each of SP1, OpenVM, Pico, and Jolt superlative on some workload class; cycle counts differ, guests differ, and GPU/CPU configurations differ. The correct posture for LZX is the one already in place - a self-hosted, median-of-runs, pure-std reproducible matrix - extended with the standard guest set (fibonacci, sha2, keccak, ed25519-verify, recursive-verify) so that LZX numbers are at least guest-comparable with the public harnesses.

Chapter 4 - LambdaClass, ZisK, and the GPU-Native Prover Roadmap

4.1 What LambdaClass is actually building

LambdaClass is an engineering firm whose public work spans the ethrex Ethereum execution client, the lambdaworks cryptography library, and the ZisK zkVM. The deployed fact: the LambdaClass team deployed its prover using ZisK on Aligned Layer in December 2025, and ethrex integrates multiple zkVM backends (SP1, RISC Zero, ZisK, OpenVM) for proving Ethereum block execution, with a Least Authority audit of ethrex (v9.0.0) reporting 20 issues and 8 suggestions. The lambdaworks 2026 roadmap (published February 2026) states the priorities explicitly: continue zero-knowledge virtual machine work, add GPU support for lambdaworks across CUDA, Metal, and WebGPU, and continue performance improvements. ZisK itself is an open-source zkVM focused on real-time proving, from the school of Jorge Baylina (co-designer of the Cairo language and its STARK proving stack): the design taste is to write the prover as a GPU program - the protocol is chosen so that its heavy layers are massively parallel, memory-coalesced kernels.

4.2 The lessons that transfer to LZX

Lesson one: choose protocols that are GPU-shaped. ZisK's design taste confirms a rule that LZX's own kernel numbers already hint at: the 270x NTT and 436x MAC kernel speedups in the labinius backend are exactly the shapes GPUs reward (regular butterflies, batched matrix-vector products, no data-dependent branches). The Ajtai commitment MAC is a matrix-multiply-accumulate over small limbs; the negacyclic NTT is a radix butterfly; both map onto tensor cores and shared-memory hierarchies. The SALSAA and sumcheck layers are linear passes over trace arrays. In other words, **the lattice stack is more GPU-shaped than the elliptic-curve stack** (which is bottlenecked on MSMs over big-field points), and LZX should exploit that consciously rather than treating GPU as an afterthought.

Lesson two: proving-network integration is a product surface. Aligned-style aggregation layers (verify many proofs cheaply, settle one commitment on-chain) are how GPU provers reach users economically. LZX's equivalent is native: the PCD and accumulation machinery (2026/289, 2026/538) plus the batched-compact envelope means shard proofs can be accumulated rather than individually verified, which is precisely the aggregation-layer value proposition but as a protocol feature. Chapter 5 item P-7 turns this into a concrete deliverable: a sharded prover whose shard proofs fold into one accumulator and one decider proof.

Lesson three: multi-backend discipline. Ethrex's multi-zkVM integration exists because different workloads favor different machines. LZX should keep its backend boundary (the PcsBackend trait and the envelope's versioned capability caps) strict enough that a future elliptic-curve or hash-based backend can sit beside the lattice one for precompile-heavy guests, without weakening the post-quantum story for the default path.

Lesson four: audit posture precedes performance claims. The ZisK/ethrex orbit publishes audits alongside performance posts; the field's incidents (chapter 3.2) make that the norm. For LZX the analogous bar is: no external performance or security claim until the estimator-gated production parameters (Wave 8.8) and the second-level fold (Stage 5.2) land, both of which are prerequisites, not enhancements.

Chapter 5 - The Next Papers: A Prioritized Implementation Pipeline

This chapter is the direct answer to the research mandate: which papers and components should be implemented next, ranked by their effect on prover time, verifier time, proof size, privacy, distributed proving, and soundness. The repo's own 831-line gap analysis (NEXT_STEPS.md) already prices most of the in-tree backlog; this chapter integrates it with the external literature and re-ranks everything against the six objectives. The headline conclusion deserves emphasis: **for the cryptographic layer, the correct next papers are mostly the completion of partial implementations already in the tree**, because the repo got ahead of the plan during Waves 6 and 7. Only three genuinely new external constructions earn a slot: the distributed-SNARK-via-folding blueprint (ePrint 2026/1995), the GPU tensor-core prover techniques (the 2024-2026 line on repurposing tensor cores for zero-knowledge workloads), and - as an optional strategic hedge - a Binius-style binary-tower backend for the fetch/bytecode layer where the data is already binary.

5.1 Prover and verifier time

The prover's time is spent in five places, in descending order at kernel scale: the sumcheck legs (field multiplications over the trace), the Ajtai commitment MACs, the NTT layer, the transcript hashing, and the fold algebra. The paper-level levers, in priority order: **SALSAA D3-D6** - the Pi-batch-star row-count-preserving batch folding (Theorem 4 of the SALSAA paper) collapses many norm statements into one sumcheck with no row growth; the fold/split/join/b-decomposition protocol set; and the engine constant-factor pass (single-binding rounds, in-place fix_variables, the Gruen and Dao-Thaler round-message optimizations). The repo's own measured baseline for the paper is 10.61 s prove at a 2^{28} witness with 41 ms verify, and the engine pass plus AVX-512 IFMA kernels is the credible 3-5x on top. **The sumcheck speedup line is already landed** (2025/1117 + 2026/587: window prover, multiproduct engine, split-eq, byte-identical transcripts) and contributed the measured 2.9x. **The sparse prover (T-and-S 6.3/7)** is the asymptotic lever: dense one-hot materialization costs $O(K \times T \log(KT))$; the sparse form costs $O(T \log T + K)$ with zero rows free, which is the difference between 2^{12} and 2^{30} scale on the same machine. Verifier time is already in the 200-300 ms class at kernel scale and is dominated by leg replay; the leg batching already cut it, and the amortized decider (Accordion-style, one decide per batch) is the remaining structural win.

5.2 Proof size

The 50-100 KB envelope is already met at kernel scale (33.0 KB batched-compact) and paper-accounted at 2^{30} (Greyhound 53 KB). The remaining size work is compositional: **the second-level fold (Stage 5.2)** removes the $k=4$ hardening's 2x column-commitment overhead while restoring binding to the estimator's required level - a net size win with a security precondition; **the chunked-chain recursion encoding (Wave 8.9)** shares the recursion phi across chunked constraint chains with carry gadgets, cutting the

labinius constraint count and with it the per-chunk proof overhead; and **rANS coding (landed)** already compresses the response layer. Akita's published 61-70 KB folded-once proofs are the honest comparable for the PCS layer, and LZX's compact openings are already inside that class. The strategic rule: no size optimization may buy bytes with soundness - the estimator runs before every merge that touches fold parameters, which is exactly the discipline the MSIS table verdict forced.

5.3 Privacy

The privacy story is a wiring problem, not a missing-paper problem. The tree contains: LatticeBlindFold (2026/1857) - the complete blinding stack with the never-opened Libra-style mask, the componentwise ABDLOP commit-and-prove, the Rej1/Rej2 samplers, and the consolidated 116/121-bit error budgets; the ZK-PCD accumulation stack (2026/289) with the CFS17/XZZ+19 zero-knowledge sumcheck and the KS24 point-update; the zk sumcheck with statement-derived masks (SALSA lineage) and the chi-square machine-checked simulators; and Greyhound's own 4.5 hiding/HVZK. What is missing is **Wave 8.6: wiring the ZK layer into the zkVM** - per-stage blinding on the Blindfold pattern, sparse masking of the one-hot columns, and composed simulator KATs proving the whole pipeline (not just each gadget) is zero-knowledge. The correct sequencing is after the sparse prover, because blinding a sparse matrix and blinding a dense one are different engineering problems, and doing them in the wrong order means doing them twice.

5.4 Distributed proving, IVC, PCD, folding

Distributed proving decomposes into sharding (many provers, one statement) and aggregation (many statements, one proof). For sharding, the trace itself shards by cycle range; each shard proves its segment against boundary claims; the boundary claims fold. The in-tree machinery for the fold is already strong: CauchyFold (2026/2011) provides residue-optimal high-arity folding (the 19-root node protocol with the certified D46 challenges), Symphony provides mu-ary one-shot folding with full subset cross-term bookkeeping, and the holography-accumulation PCD (2026/538) provides the non-uniform decider for heterogeneous shard shapes. The one genuinely new paper to implement is **ePrint 2026/1995 (Distributed SNARK via folding schemes)** as the composition blueprint - it is precisely the pattern of many distributed provers folding into one accumulator with communication proportional to the accumulator, and it slots onto the existing PCD drivers. For aggregation, the ZK-PCD accumulation (2026/289) and the Quasar partial-evaluation accumulation are the in-tree levers, with Quasar Q2/Q3 (the multicast union polynomial and the 2-to-1 fold) as the designated L-effort completion. IVC in the Nova sense is subsumed by the folding-plus-decider pattern; the repo's PCD drivers already express the IVC loop as an accumulation chain, which is the modern formulation and the one to keep.

5.5 Soundness, completeness, security

Chapter 10 is the full program; the paper-level items that belong in the pipeline are: **production parameter instantiation (Wave 8.8)** - digest-pinned SecurityProfiles whose every (n, k, q, modulus class, challenge distribution) tuple passes the SIS estimator at the claimed level, replacing the illustrative toy parameters that the README honestly flags; **the second-level fold (Stage 5.2)** to close the 2^{12} -binding gap the estimator exposed; **QROM composition attestations (Wave 8.7)** regenerated from the real stage list with machine-checked grinding bounds; and **completeness hardening as a test discipline** - every protocol module ships adversarial negative tests (tamper suites, forged-triple rejection, wraparound refusal) in the same commit as its identity tests, which is how the repo caught the Section-4 lookup attacks and the vacuous-gate defect.

5.6 The priority table

#	Item	Source	Categori es	Effor t	Expected effect
P1	Sparse one-hot prover (0s are free)	T&S; 2025/105 sections 6.3/7; jolt read-write-matrix design	E, D	L	removes the 2^{12} cap; $O(KT)$ to $O(T + K)$; the scalability unlock
P2	Instruction-semantics constraint families	T&S;/zkVM substrate (constraints.rs)	E, S	M	full RV64IMAC proving without re-execution
P3	SALSAA D3 + D5 + D6	SALSAA 2025/2124 Theorem 4 + zem blueprint	E	M	3-5x sumcheck layer; smaller norm proofs
P4	Second-level fold (Stage 5.2)	DESIGN_50KB + SECURITY.md	S, E	M	restores estimator-grade binding; halves column commitments
P5	Goldilocks AVX-512/IFMA kernels	Wave 8.2	E	M	4-8x on field-bound stages
P6	lattice-rns 8-prime crate	Wave 8.3	E	M	10-50x on LaBRADOR polx path
P7	Sharded distributed proving	ePrint 2026/1995 + CauchyFold + holography PCD	D, E	L	near-linear prover scaling across machines
P8	ZK wiring (Wave 8.6)	LatticeBlindFold 2026/1857	P	M	the zkVM becomes actually zero-knowledge
P9	Production parameters + QROM attestations	Waves 8.7/8.8	S	M	honest 128-bit posture; audit-ready
P10	GPU prover phase 1 (NTT + MAC kernels)	tensor-core ZK literature; chapter 8	E, D	L	10-40x over scalar on the two dominant kernels
P11	Quasar Q2/Q3 + Symphony CP-SNARK + Akita planner	in-tree designated 8.10	E, D	L	lookup aggregation and PCS production polish
P12	Chunked-chain recursion (8.9)	labinius constraint-count program	E	M	recursion overhead down; depth control

Table 5.1 - Prioritized implementation pipeline (E = efficiency, P = privacy, S = security, D = distributed)

Two sequencing rules govern the table. First, soundness preconditions (P4, P9) precede the performance items that would otherwise be redone under different parameters - the $k=4$ hardening is the standing example of parameters changing proof structure mid-optimization. Second, the sparse prover (P1) precedes privacy wiring (P8) and GPU work (P10), because both consume the sparse representation: blinding a sparse matrix is a different construction than blinding a dense one, and GPU kernels written against dense layouts would be rewritten. The roadmap in chapter 12 respects both rules.

Chapter 6 - Hitting 100 Million Cycles per Second

The target - 100 million RISC-V cycles per second of **proven** throughput - deserves an explicit budget rather than a hope. One hundred million cycles per second is 2^{30} cycles in 10.7 seconds. No CPU zkVM publicly demonstrates that today on general ISA semantics; the credible path is a stack: the sparse prover removes the asymptotic wall, SIMD and RNS remove the constant-factor wall on CPU, the GPU removes the kernel wall, and sharding multiplies throughput across machines. Each stage of that stack is independently measurable, which is what makes the target an engineering program instead of a leap.

6.1 Where the cycles go

Per proven cycle, the memory argument's cost is dominated by the one-hot layer: each cycle touches 9 instances (4 register limbs, 4 RAM limbs, 1 fetch) with 13 sumcheck legs each, and the dense path materializes $K \times T_s$ one-hot matrices. The paper-faithful sparse form replaces every zero run with an index-list factor, makes the per-register eq arrays precomputed lookup tables, and evaluates the one-hot constraints in $O(T)$ with an $O(\log T)$ or $O(K)$ term for the table side. After the sparse transformation, the prover's per-cycle work collapses to: approximately 120 field multiplications per leg-batch round across the 12 staged sumchecks (already measured), the Ajtai MAC over the non-zero one-hot entries (pay-per-bit, zero-skipping), and the transcript hash per round. That is the regime in which SIMD and GPU constants matter.

Stage	Mechanism	Cycles/sec class	Basis
Today (dense, kernel scale)	dense one-hots, scalar-competent engine	$\sim 10^5 - 10^6$	measured: 185-451 cycles in 1.4-4.6 s including overhead
+ Sparse prover (P1)	zero entries free; index-list factors; eq tables	$\sim 10^6 - 10^7$	T&S; paper asymptotics; removes O(K) per-entry cost
+ AVX-512/IFMA + RNS (P5, P6)	field kernels 4-8x; polx 10-50x	$\sim 10^7$	labinius 49x round precedent; kernel-level 270x/436x
+ GPU phase 1-2 (P10)	tensor-core MAC + NTT kernels, CUDA graphs	$\sim 3 \times 10^7 - 10^8$	tensor-core ZK literature: order-of-magnitude on MSM-class kernels
+ Sharding (P7)	S machines folding shard proofs	S x per-machine	2026/1995 communication profile; accumulation decider
+ Precompiles	guest-program class shortcuts	workload-dependent	SP1/OpenVM precompile economics; only for hot guest ops

Table 6.1 - Throughput budget stack (per-machine, general ISA semantics)

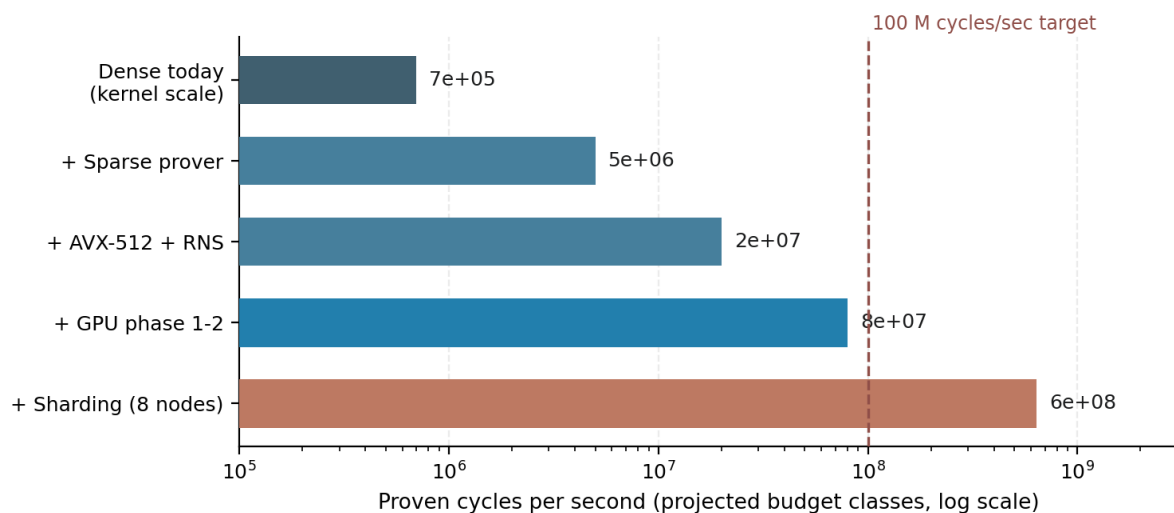


Figure 6.1 - Projected cumulative throughput budget by stack stage (log scale; classes from Table 6.1, single machine except the sharding row). The 100 M target line is crossed only in the GPU-plus-sharding regime.

Read the table as cumulative and conditional: the sparse prover is the gate for every number below its row, and the GPU row's range assumes the kernel budget below lands. The honest current answer for **CPU-only** LZX after Waves 8.2/8.3 is the 10^7 class - between 5 and 20 million cycles per second on a strong AVX-512 machine, which is already in the SP1/Jolt public class on comparable guests. The 10^8 class requires the GPU row plus either sharding or precompile-heavy guests. A single 4090-class GPU delivers on the order of 10^{14} INT32 MACs per second across its tensor-friendly paths; the Ajtai MAC layer at 2^{30} cycles times roughly 64 column limbs is about 7×10^{13} MACs if laid out as batched matrix products - one second of tensor-core time, before memory traffic, which is the real cost. The budget is therefore not arithmetic-bound but bandwidth-bound: the kernel design in chapter 8 exists to keep the MAC layer coalesced and the NTT layer shared-memory-resident.

6.2 The measurement discipline

Throughput claims will be made against the reproducible harness (median-of-runs, pure-std) with the standard guest set, and the headline number will be the memory-argument prover at 2^{24} and 2^{26} cycles, not a kernel microbenchmark. Two explicit gates are added to the harness: a cycles-per-second column derived from wall-clock over executor-verified cycle counts, and a saturation report per stage (what fraction of time is hashing, sumcheck, MAC, NTT, fold) so that every optimization targets the measured top of the stack rather than the imagined one. The 2025-2026 zkVM benchmarking literature's core criticism - non-comparable cycle accounting - is answered by pinning guest programs, cycle definitions, and hardware in the harness itself.

Chapter 7 - Hitting the 50-100 KB Proof Envelope

The proof-size target is the one LZX is closest to meeting, and the composition data explains both why and what remains. At kernel scale the batched-compact pipeline delivers 33.0 KB on fibonacci: 6 KB of staged sumcheck legs (the 12-batch restructure collapsed 55 KB across 108 legs), 3.5 KB of values-only claims, 16 KB of column commitments after the $k=4$ hardening, under 1 KB of carriers, 6 KB of compact openings, and 0.5 KB of statement. At full scale, the Greyhound engine's Table 4 accounting holds evaluation proofs at 53 KB for $N = 2^{30}$, and the 2^{26} -scale sub-proof is exercised in the benchmark harness (GREYHOUND_230=1 runs the 2^{30} statement). The remaining distance between the kernel-scale 33 KB and a system-scale 50-100 KB at 2^{30} is exactly the recursion and aggregation machinery that connects the two: the memory-argument proof at scale, folded once into the Greyhound-class carrier, with the chunked-chain encoding holding down the per-chunk constraint overhead.

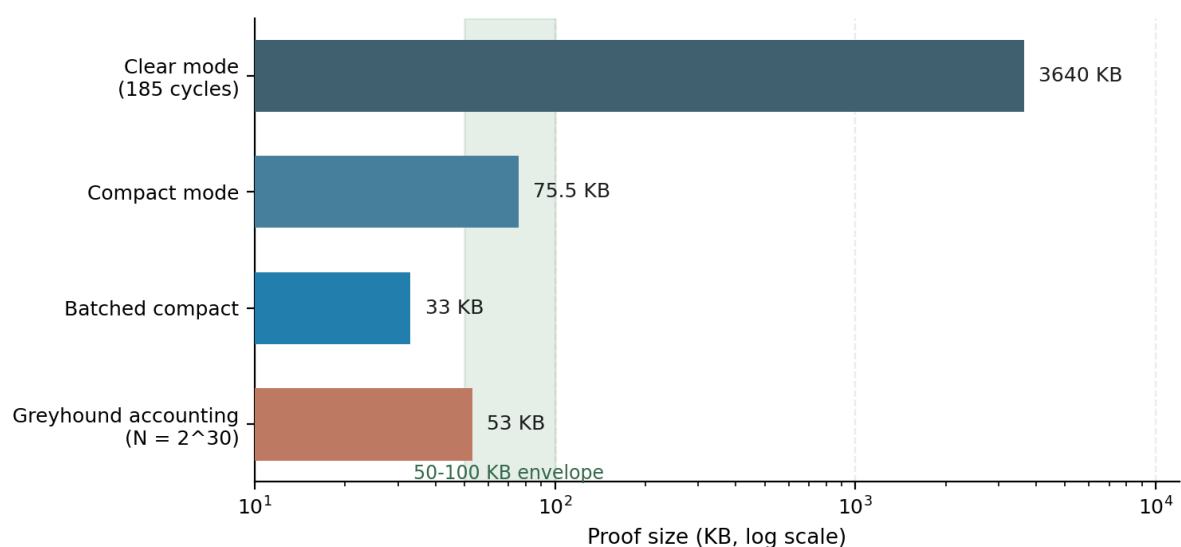


Figure 7.1 - Proof-size trajectory: clear mode to batched-compact (kernel scale, fibonacci), with the Greyhound 2^{30} accounting and the 50-100 KB envelope for scale. Log scale.

Four size levers remain, all compositional rather than new cryptography. **First, the second-level fold:** the $k=4$ interim hardening doubled the column commitments (16 KB of the 33); the two-level fold restores estimator-grade binding while retiring the doubling, for a net size improvement of roughly 20-30 percent with the security precondition attached. **Second, the claims layer:** 3.5 KB of values-only claims is already re-derived by the verifier's leg replay; extending the values-only discipline to the boundary claims of sharded proofs is the sharding size lever. **Third, recursion encoding (Wave 8.9):** shared-phi chunked chains with carry gadgets cut the labinus constraint count per chunk, which multiplies through the per-chunk proof overhead at depth. **Fourth, the response codec:** rANS coding is landed for the opening layer; the same entropy discipline applies to the digit tables wherever they survive the compact openings. The envelope's floor is structural: the PCS opening at the 2^{30} scale is a fixed cost on the order of tens of KB (53 KB accounted), so the system target is correctly stated as 50-100 KB **total** for the full statement, of which the PCS layer is the majority and the memory argument the minority - the inverse of the pre-compact pipeline, where 95 percent of the proof was the opening machinery.

The verification-time corollary: the 50-100 KB envelope is only worth meeting if verification stays in the sub-second class. The measured 196-295 ms at kernel scale is dominated by the verifier's leg replay and public-table recomputation (the program image and initial memory image, which the verifier must read regardless). At 2^{30} , leg replay stays polylog in the cycle count but public-image recomputation grows linearly with the program - which is the correct economics (public inputs are paid once by the verifier, private witness never), and the amortized decider (Accordion pattern, one decide per batch) is the structural cap on the rest.

Chapter 8 - GPU Acceleration in Rust

8.1 Naming clarification first

The project brief named two candidates - **cutdsl** and **cudaoxide** - and both need honest identification before an architecture can be chosen. CUTLASS DSL (CuTe-DSL) is NVIDIA's Python-embedded domain-specific language for writing CuTe kernels on top of the CUTLASS/CuTe C++ template ecosystem; it is Python-first and has no Rust story, so it is not the vehicle for a Rust GPU prover - though its kernel *designs* (tiled MMA pipelines, pipeline stage planning) are exactly the documents to read when writing the equivalent PTX or CUDA C. **cudaoxide** maps to the real Rust ecosystem option: **cudarc**, the maintained safe-ish Rust interface to the CUDA driver API with PTX compilation and module loading, used in production by several ZK projects. The alternative families are: **wgpu/WebGPU** (portable across vendors, compute-shader model, weaker control of tensor cores), **rust-gpu** (writing kernels in Rust itself via the Vulkan/SPIR-V backend - attractive for language purity, still immature for production kernels), and raw bindings. The recommendation is unambiguous: **cudarc as the backend, CUDA C kernels compiled to PTX at build time, with a CPU fallback behind a GpuBackend trait** - preserving the repo's zero-external-runtime discipline where the same binary runs unchanged without a GPU, mirroring

the runtime-detection pattern the AVX-512 backends already use.

8.2 The kernel program

The GPU program is deliberately small: three kernel families cover the measured top of the prover's stack. **Kernel family 1: the Ajtai commitment MAC.** The MAC is a matrix-vector accumulate over small limbs (the labinius vpmaddwd discipline); on GPU it becomes a tiled matrix multiply-accumulate over the 16-bit limb domain, which is the tensor-core (mma.sync / wmma) shape - the 2024-2026 literature on repurposing tensor cores for zero-knowledge workloads reports order-of-magnitude gains precisely on this class, and the repo's own 436x CPU kernel-level result on the same algebra confirms the shape is regular enough to exploit. **Kernel family 2: the negacyclic NTT.** Radix butterflies with Twiddle-factor tables in shared memory, a batched transposition between stages, and the psi-scaling folded into the twiddles; the 270x CPU kernel result at batch-of-32 defines the reference semantics to match bit-exactly. **Kernel family 3: the sumcheck round kernels.** The per-round bind-and-reduce over the virtual-polynomial arrays is a linear pass - coalesced reads, no branches, one reduce per round message - and the eq/eq-table precomputation is a prefix-scan shape. Everything else (fold algebra, transcript hashing) stays on CPU initially; Keccak-f1600 on GPU is deliberately deferred because the substrate's CPU implementation is bit-exact and the transcript is a serial dependency by construction.

8.3 Architecture and integration discipline

Phase	Content	Expected effect	Prerequisites
GPU-1	cudarc backend, NTT + MAC kernels, bit-exactness tests vs scalar reference, CUDA graphs + streams	10-40x on the two dominant kernels vs scalar; 3-8x vs AVX-512	sparse prover (P1) so kernels see sparse layouts
GPU-2	tensor-core MAC tiling (mma.sync paths), sumcheck round kernels, pinned-memory pipeline, saturation report in the harness	prover top-of-stack dominated by GPU; 10^7 - 10^8 cycles/sec class reachable	GPU-1 lands with bit-exact KATs
GPU-3	multi-GPU sharded prover (one GPU per shard), fold on host, accumulation decider on GPU	near-linear scaling with shard count	P7 sharding architecture

Table 8.1 - GPU acceleration plan (phased)

The integration discipline is the repo's existing one, extended: every GPU kernel lands with (a) a bit-exactness test against the scalar reference (the same standard that validated the AVX-512 backends), (b) runtime detection with automatic fallback, (c) no new external runtime dependencies in the default build - the CUDA path activates only when the driver is present, and (d) a measured stage-saturation report added to the benchmark matrix so that the next kernel decision is data-driven. Kernels are written CUDA C, compiled to PTX at build time and embedded; cudarc loads and launches them; a thin GpuBackend trait keeps the door open for a wgpu second backend later without touching protocol code. This is the same architecture ZisK validated from the other direction (GPU-first protocol choice), applied to a stack that - as chapter 4 argued - is already more GPU-shaped than the curve-based alternatives.

Chapter 9 - Formal Verification: Bend, HVM, and What They Can and Cannot Do

9.1 What Bend actually is

The project brief described Bend as a new formal verification system, and the report owes the project an honest correction before architecture is built on it. **Bend (HigherOrderCO) is a high-level functional programming language that compiles to HVM (the Higher-order Virtual Machine), an optimally-evaluated, massively parallel runtime written in Rust.** Bend's guarantees are about *execution*: programs are automatically parallelized across cores and GPUs by the runtime's optimal evaluation semantics, without threads or explicit data parallelism. Bend is **not** a proof assistant: it has no theorem-proving kernel, no tactic language, and no mechanical proof checking. The systems that provide machine-checked theorems are Lean 4, Coq, Isabelle/HOL, and Agda - that is where theorems like 'the fold identity holds for all challenges' or 'the norm budget never wraps the modulus' get proved once and checked forever.

This distinction does not demote Bend's value to this project; it relocates it. A zkVM has exactly the problem Bend and HVM are good at: **independent, executable specifications that must run fast enough to be tested at scale.** The repo already maintains two independent implementations of the RV64IMAC semantics (the traced executor and the byte-level reference interpreter) and runs 131-program differential corpora; a third implementation in Bend would run the same corpora on the HVM runtime with automatic parallelism, at a level of independence from the Rust codebase that a Rust reimplement cannot claim (different language, different runtime, different evaluation semantics). The same argument applies to the algebraic laws: a Bend encoding of the ring/NTT/fold identities executed on randomized inputs is property-based testing with an independent oracle, which is precisely the class of assurance that catches the field's recurring under-constrained-circuit bugs.

9.2 The two-track program

Track	Tool	Content	Phase
Executable specification	Bend + HVM	RV64IMAC reference executor in Bend; differential corpus vs the two Rust interpreters; algebra-law property battery (ring, NTT, fold identities, norm budgets)	F1 (parallel differential oracle)
Executable specification	Bend + HVM	Guest-program generator in Bend for the fuzz harness; randomized trace/tamper generation at HVM speed	F2
Machine-checked proofs	Lean 4	Statements (not full proofs) for the 12 soundness-critical lemma classes: fold identities, challenge-space operator-norm bounds, no-wraparound norm gates, MSIS estimator inequalities	F3a
Machine-checked proofs	Lean 4	Full proofs for the 2-3 highest-leverage lemmas (the fold identity family and the no-wraparound condition), with Rust-side KATs cross-checking the proved statements	F3b
Integration	CI	Bend differential suite and Lean statement checks wired into the test matrix; every protocol module lands with its negative tests	F4

Table 9.1 - Formal verification program (two tracks, three phases)

The phasing is chosen so that each step pays for itself before the next: F1 gives immediately stronger conformance testing (the RVC funct3 mapping defect class that differential testing cannot catch by construction is at least partially covered by a third, independently-designed mapping); F2 scales the fuzz corpus; F3a costs days and creates the proof surface; F3b is where proof engineering effort concentrates, on the lemmas whose falsehood would be soundness-critical rather than performance-relevant. The Valida project's formally specified ISA is the field's proof that this layer pays; the zkVM incident literature (chapter 3) is the proof that it is needed. The honest scoping rule: **Bend for what runs, Lean for what is true**, and no conflation of the two.

Chapter 10 - Soundness, Completeness, and Security Program

The security program's premise is the field's hardest lesson: zkVM incidents in 2024-2026 (the Jolt findings by ZK Security, the 20-issue RISC Zero audit by Veridise, the recurring classes in the zkVM SoK) were overwhelmingly **implementation and parameterization failures, not broken assumptions**. LZX's substrate is already disciplined - hard norm gates that refuse to fold rather than wrap silently (wraparound mod q destroys SIS binding), paper-calibrated ring-element challenge distributions with certified operator-norm bounds, the committed Quasar lookup path with statement-bound τ , a zero-dependency SIS security estimator pricing ADPS16/BDGL16/LGSA attack paths, QROM query ledgers with worst-case rejection amplification, digest-pinned KATs, envelope fuzzing, and machine-checked HVZK simulators. The program below completes it.

10.1 The five remaining gaps, in order

G1 - production parameters (Wave 8.8). The README states it plainly: parameter sets are illustrative and must be reviewed before production use. The completion is estimator-gated, digest-pinned SecurityProfiles: every profile fixes (n, k, q, modulus class, challenge distribution, fold depth, response length) and refuses to construct if the estimator's verdict at the claimed security level fails. The profiles are versioned in the envelope's capability caps so proofs carry their security level explicitly. **G2 - the second-level fold (Stage 5.2).** The estimator's honest verdict on the single-level $k=2$ fold was approximately 2^{12} binding; $k=4$ with amplitude 2^6 is the shipped interim. The two-level fold is the sound posture and a size win; it is the single most security-relevant protocol item. **G3 - QROM attestations from the real stage list (Wave 8.7).** The query ledger framework exists; the attestations must be regenerated against the production pipeline's actual stage list with machine-checked grinding bounds. **G4 - completeness-as-testing.** Every module lands with negative tests in the same commit (the discipline that caught the Section-4 lookup attacks); the target is that every soundness-relevant gate has a test proving it rejects. **G5 - RVC standard conformance.** The funct3 mapping is codebase-local and mirrored by the reference, so differential testing cannot catch a conformance bug by construction; the fix is conformance vectors from the RISC-V specification suite plus the Bend third implementation (chapter 9).

Risk	Control	Status
Challenge-space deficit (knowledge error $2^{8..2^31}$ vs papers' 2^{100+})	fixed-weight ternary ring challenges with op-norm rejection (lattice-core::short_challenge)	landed (W6)
Norm wraparound silently destroying SIS binding	hard norm gates $\beta < \min(q/2, \beta^*)$ on every folding module	landed (W6)
Uncommitted lookup tables (verifier binds nothing)	committed Quasar Q1: tau from commitments, counting-map difference, forged-triple rejection	landed (W6)
Toy SIS parameters presented as secure	zero-dependency SIS estimator + SecurityProfiles	estimator landed; profiles are Wave 8.8
Fiat-Shamir composition errors	QROM query ledger + attestations + composition review	framework landed; real-stage-list regeneration is Wave 8.7
Compact-fold binding below claim	estimator-run MSIS table + $k=4$ interim hardening	interim shipped; second-level fold outstanding (G2)
ISA conformance drift	differential corpus + golden vectors + RISC-V conformance vectors + Bend third implementation	corpus landed; conformance suite is G5
ZK claims without machine-checked simulators	HVZK simulator + chi-square KATs; distributional simulator for blinded sumcheck	landed for gadgets; zkVM-wide composition KATs are Wave 8.6

Table 10.1 - Risk-to-control map (status after Waves 6-7)

10.2 Audit readiness

The external-audit bar (the field's norm after the Jolt and RISC Zero audits) is: a frozen parameter profile with estimator verdicts, a protocol document per pipeline stage (the repo's docs/papers tree is most of the way there), tamper suites demonstrating rejection for every gate, and a stated threat model distinguishing ROM from QROM claims. The repo's four fixed-vulnerability post-mortems in SECURITY.md are the right artifact to extend rather than a liability to hide: they demonstrate the detection-and-fix loop works. The sequencing rule from chapter 5 applies here with force: G1 and G2 precede the performance waves, so that no benchmark published externally is later invalidated by a parameter change.

Chapter 11 - Immediate Implementation: Completing SALSAA and Twist-and-Shout

This chapter is the working plan for the next implementation session, scoped to the two papers the project has designated as the immediate completion targets. Both are partial: SALSAA has its D1+D2 and the A2-A5 response-layer stack landed (the ring norm sumcheck Pi_{norm} , Pi_{bin} , the staircase RoK, the VDF binary staircase, committed-AIR plus folding, and the D4 swap consumed by the Akita PCS), and the zkVM memory argument proves and verifies without re-execution after Wave 7.4's core (552 workspace tests). What remains for each is precisely specified below, with the file-level landing zones and the test discipline per item.

11.1 SALSAA completion items

Item	Content	Landing zone	Tests
D3 - Pi-batch-star	row-count-preserving batch folding (Theorem 4): fold m norm statements into one sumcheck without row growth; random-power RLC of the claims	lattice-sumcheck/src/batch.rs (extend <code>prove_batch</code>)	identity + row-count invariance + tamper
D5 - fold/split/join/b-decomp	the protocol set from the paper's applications section: statement folding, witness splitting, batch joining, and the b-adic decomposition for norm statements; port structure from the zem reference, not its anti-patterns	lattice-salsa/src/salsaa.rs (new module <code>protocols.rs</code>)	each protocol: identity, negative, norm-budget regression
D6 - engine constant-factor pass	single-binding rounds (bind once per round, not per evaluation point), in-place <code>fix_variables</code> (no per-round clones), Gruen/Dao-Thaler round-message optimizations	lattice-sumcheck/src/{sumcheck, virtual_poly}.rs	byte-identical transcripts vs current engine (the repo's established standard)
D7 - QROM registration	register the SALSAA stages in the QROM domain registry; composition review for the response-layer swap	lattice-qrom/src/domains.rs	attestation digest KAT

Table 11.1 - SALSAA remaining backlog (crate: `lattice-salsa` + consumers)

D3 is the highest-value item internally: the staged leg batching in the zkVM (legbatch.rs) already uses a random-power RLC via prove_batch, and Pi-batch-star's row-count preservation is the property that lets the batched legs stay 12 sumchecks at any program size instead of growing with instance count - which is why it is also on the critical path of the sharding architecture (P7). D6 is mechanical but system-wide: the engine's sum_products re-does factor half-binding per evaluation point (an approximately 2x redundancy measured in NEXT_STEPS), and fix_variables clones full arrays per round; both are fixable without touching round-message semantics, and the byte-identical-transcript test standard already in the repo is exactly the right regression harness.

11.2 Twist-and-Shout completion items

Item	Content	Landing zone	Tests
T1 - sparse one-hot prover	index-list one-hot factors (zero rows free), per-register eq arrays, Eq-46 lookup tables, the cycle-major/address-major layouts from the jolt read-write-matrix design; removes the $K \times T_s$ materialization	lattice-memory/src/sparse_engine.rs (extend) + lattice-zkvm/src/memproof.rs	bit-identical round messages vs dense path at kernel scale; then dense-path retirement behind a flag
T2 - instruction-semantics constraints	the arith/logic/comparison/control/routing/halted constraint families over the column substrate: decode-consume, ALU limb arithmetic, branch/jump routing, halt propagation	lattice-zkvm/src/constraint s.rs (substrate staged)	per-family: positive on golden vectors, negative on tampered traces; the differential corpus as ground truth
T3 - zero-skipping Ajtai + bit-packed one-hot columns	commit only the non-zero one-hot entries (pay-per-bit), bit-packing the one-hot columns instead of 22-bit limbs (32-64x denser)	lattice-commitment/src/sparse.rs + lattice-zkvm/src/columns.rs	commitment KAT + norm-budget regression
T4 - locality-aware binding orders	section 8.2: 3i multiplications for 2^i -local reads by binding address variables before time variables in the locality-aware order	lattice-zkvm/src/legbatch.rs	round-count regression on locality-structured traces
T5 - non-revealing Akita opening	the amortized, non-disclosing opening replacing the full-witness reveal on the response layer (the hinge for succinctness and ZK)	lattice-akita/src/salsa_response.rs	soundness: extraction test; privacy: simulator KAT

Table 11.2 - Twist-and-Shout remaining backlog (crates: lattice-memory + lattice-zkvm)

T1 and T2 together are what turns the current memory-argument zkVM into an ISA-complete zkVM at scale: T1 removes the cycle cap (the 2^{12} limit and the 256-word RAM window are both artifacts of dense materialization), and T2 makes the prove path cover decode-execute semantics rather than memory only. The sequencing inside chapter 11 is D3/D6 first (they are pure wins with byte-identical-transcript regression harnesses already available), then T1 (the unlock), then T2 (the largest single work item, but on staged substrate with golden-vector ground truth), then T3/T4/T5 as the polish wave. The session plan follows exactly this order, and every item lands with its negative tests in the same commit, per chapter 10's G4 discipline.

Chapter 12 - Phased Roadmap and Success Criteria

The roadmap binds the pipeline (chapter 5), the engineering budgets (chapters 6-8), the verification program (chapter 9), and the security program (chapter 10) into three waves with explicit exit criteria. The waves respect the two sequencing iron rules established earlier: soundness preconditions precede performance work that parameters would invalidate, and the sparse prover precedes privacy wiring and GPU kernels that would otherwise be written twice.

Wave	Theme	Contents (in dependency order)	Exit criteria
W8 (now)	completion + CPU constants	SALSAA D3/D6/D5; T-and-S T1 (sparse prover), T2 (constraint families); second-level fold (P4); AVX-512/IFMA + RNS (P5/P6); ZK wiring (P8) after T1; production params + QROM attestations (P9)	2 ²⁰ -cycle program proved end-to-end without re-execution, under 100 KB, on one machine; estimator-passing profile; 5-20 M cycles/sec CPU class
W9 (next)	GPU + distribution	GPU-1/GPU-2 (cudarc, NTT+MAC+tensor-core MAC, sumcheck kernels); sharded prover P7 on 2026/1995 + CauchyFold + holography PCD; Quasar Q2/Q3; chunked-chain recursion (P12)	2 ²⁶ -scale program at 3 x 10 ⁷ - 10 ⁸ cycles/sec class with GPU; multi-machine sharding near-linear; proof inside the 50-100 KB envelope
W10 (maturity)	verification + network	Lean F3a/F3b proofs for the fold-identity and no-wraparound lemmas; Bend F1/F2 differential oracle in CI; multi-GPU GPU-3; proving-network integration surface; external audit preparation	audit-ready dossier; machine-checked core lemmas; third-implementation conformance; published reproducible benchmark dossier vs the standard guest set

Table 12.1 - The three-wave roadmap

12.1 Success criteria as KPIs

KPI	Now (measured)	W8 exit (target)	W9 exit (target)
Proven program scale (cycles, single machine)	2^{12} (dense cap)	2^{20}	2^{26}
Proving throughput	kernel-scale only	5-20 M cycles/sec (CPU)	3×10^7 - 10^8 cycles/sec (GPU + sharding)
Proof size (memory argument)	33.0 KB at 185 cycles	under 50 KB at 2^{20}	50-100 KB at 2^{26} - 2^{30} (envelope met at scale)
Verification time	196-295 ms (kernel scale)	under 500 ms at 2^{20}	sub-second at scale
Verifier re-executes program	no (W7.4 core)	no; full ISA semantics covered	no
Zero-knowledge	gadget-level simulators	zkVM-wide composed simulator KATs	same, under production params
Binding/soundness accounting	k=4 interim hardening	second-level fold; estimator-passing pinned profile	external-audit-grade dossier
Test corpus	1,136 tests	plus per-item negative suites; 2^{20} e2e	plus GPU bit-exactness and sharding e2e

Table 12.2 - KPI targets by wave exit

12.2 What success buys

If the waves exit on criteria, LZX occupies a position no other system in the chapter 3 table holds: a post-quantum (Module-SIS) zkVM proving 2^{26} -cycle programs at GPU-class throughput inside a 50-100 KB envelope with an estimator-accounted 128-bit posture and a machine-checked core. The Greyhound 53 KB accounting at 2^{30} is then not a paper claim carried in the README but a system property measured by the reproducible harness. That is the standard this roadmap is calibrated to, and the standard the next implementation session starts earning, one item at a time, beginning with the SALSAA and Twist-and-Shout completion plan of chapter 11.

References

- Nguyen, N. K. and van der Woerd, R. and Lyubashevsky, V. and Seiler, G. Greyhound: Fast Polynomial Commitments from Lattices. IACR ePrint 2024/1293.
- Lyubashevsky, V. and Seiler, G. Lattice-Based SNARKs and Proofs of Linear Relations. IACR ePrint 2022/1341 (LaBRADOR).
- Beck, W. and Shi, K. LaBRADOR-in-Tongues. 2026 (in-tree engine reference).
- Kuriyama, K. and Lai, R. and Osadnik, M. and Tucci, V. SALSAA(SA): Lattice-Based Sumchecks for Norm Proofs. IACR ePrint 2025/2124 (SALSAA).
- Sous, D. and Dallaire, N. and Le, T. and Tyner, J. Twist and Shout: Proving Memory and Machine Executions with One Hot Polynomials. IACR ePrint 2025/105.
- Diamandis, T. and Evans, N. and Garg, S. et al. (in-tree documentation). Ring Lookup Arguments over Lattice Rings. IACR ePrint 2026/471.
- Chen, Y. and Manning, S. LatticeFold+ Norm Controls. IACR ePrint 2026/721 (ell2-norm checks, JL projection RoK).

Tang, X. and Puthoor, I. S. ZK-PCD from Accumulation Schemes. IACR ePrint 2026/289.

Castellanos, V. and Huang, Y. PCD via Holography Accumulation. IACR ePrint 2026/538.

Roy, B. and Vaziri, S. LatticeBlindFold: A Lattice-Based Analogue of NovaBlindFold. IACR ePrint 2026/1857.

Jain, A. and Russo, R. CauchyFold: Residue-Optimal High-Arity Lattice Folding. IACR ePrint 2026/2011.

Lee, S. and Kim, J. Tensor-Train Random Projections for Shortness. IACR ePrint 2026/2146 (TTRP).

Zhou, Z. and Wang, L. Speeding Up Sum-Check Proving. IACR ePrint 2025/1117 and 2026/587 (extended).

Baldominos, A. and Merlo, S. The Monomial-Basis (Projective) Sum-Check. IACR ePrint 2026/762.

Fenzi, M. and Kalai, Y. Proving CPU Executions in Small Space. IACR ePrint 2025/611.

Ibarra, M. and Qian, Z. Neo: Lattice-Based Folding for CCS over Small Fields. IACR ePrint 2025/294.

Albrecht, M. and Scholl, P. Accordion: Revisiting the IPA-Sumcheck Connection. IACR ePrint 2025/1325.

Khabbazian, M. et al. Distributed SNARK via Folding Schemes. IACR ePrint 2026/1995.

Yang, Y. et al. SoK: Understanding zkVM - From Research to Practice. IACR ePrint 2026 (systematization).

Fenbushi Capital. Benchmarking zkVMs: Current State and Prospects. August 2025.

ZK Security. ZK/SEC Quarterly: Jolt zkVM findings. 2025-2026 series.

Veridise. RiscZero zkVM Audit Report. February 2025.

HigherOrderCO. Bend and the HVM optimal runtime (HVM2). 2024-2026.

LambdaClass. lambdaworks 2026 roadmap; ZisK zkVM; Aligned Layer deployment. 2025-2026.

Succinct Labs. SP1: a performant, 100 percent open-source zkVM. 2024-2026.

NVIDIA. CUTLASS and the CuTe DSL (kernel design references). 2024-2026.